LAPORAN PRAKTIKUM

POSTTEST 6

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Belva Abhinaya (2609106005)

Kelas (A1 ’26)

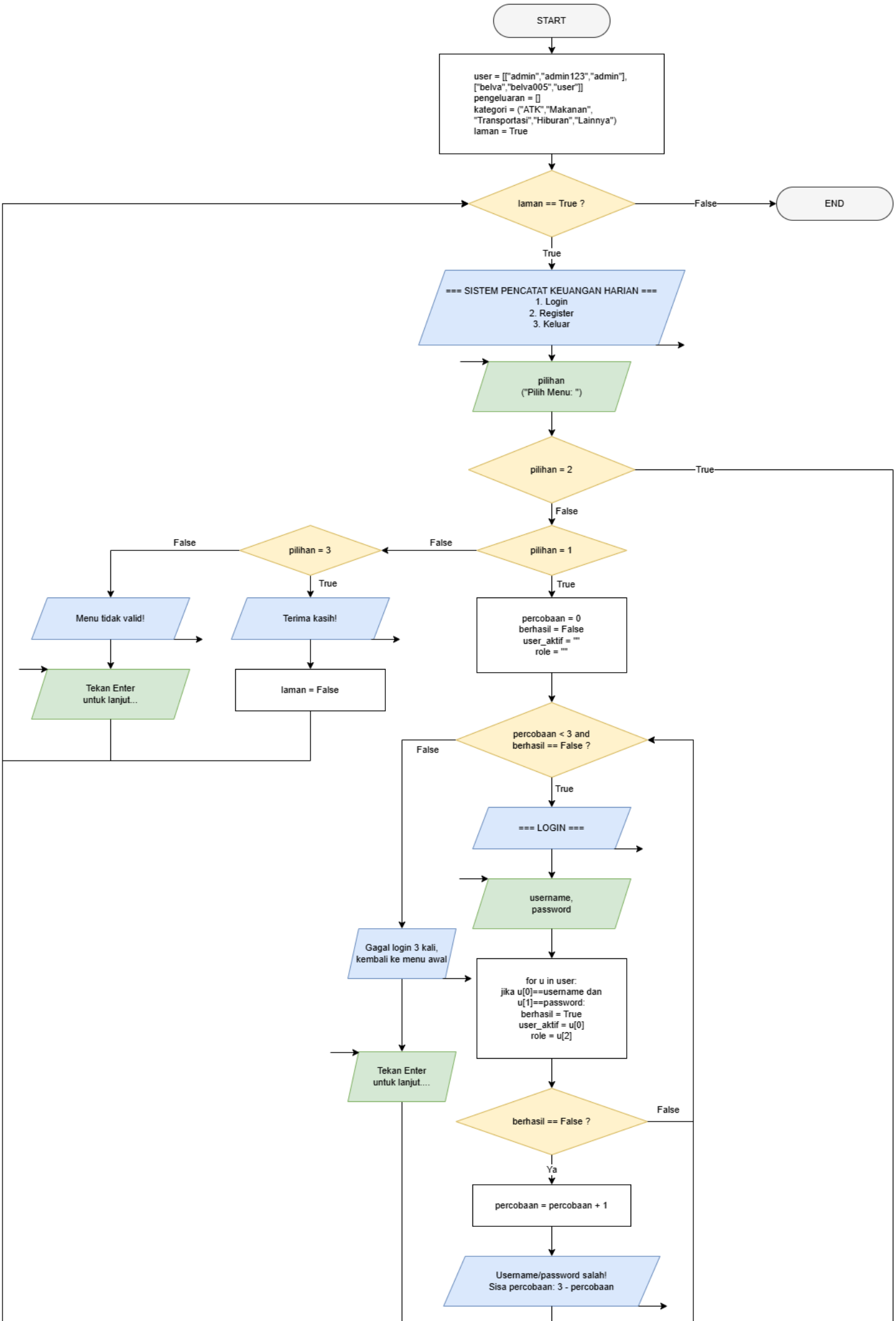
PROGRAM STUDI INFORMATIKA

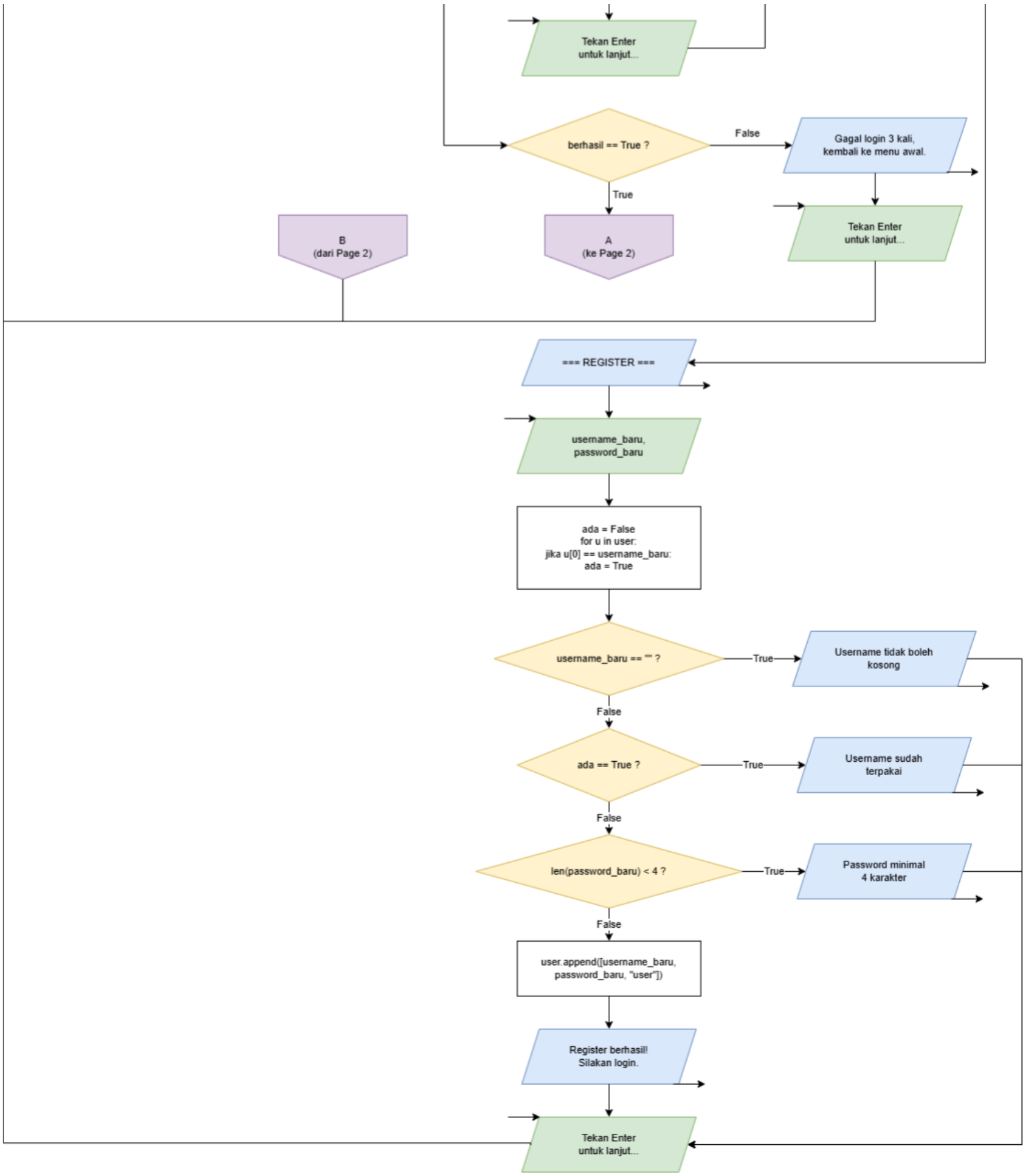
UNIVERSITAS MULAWARMAN

SAMARINDA

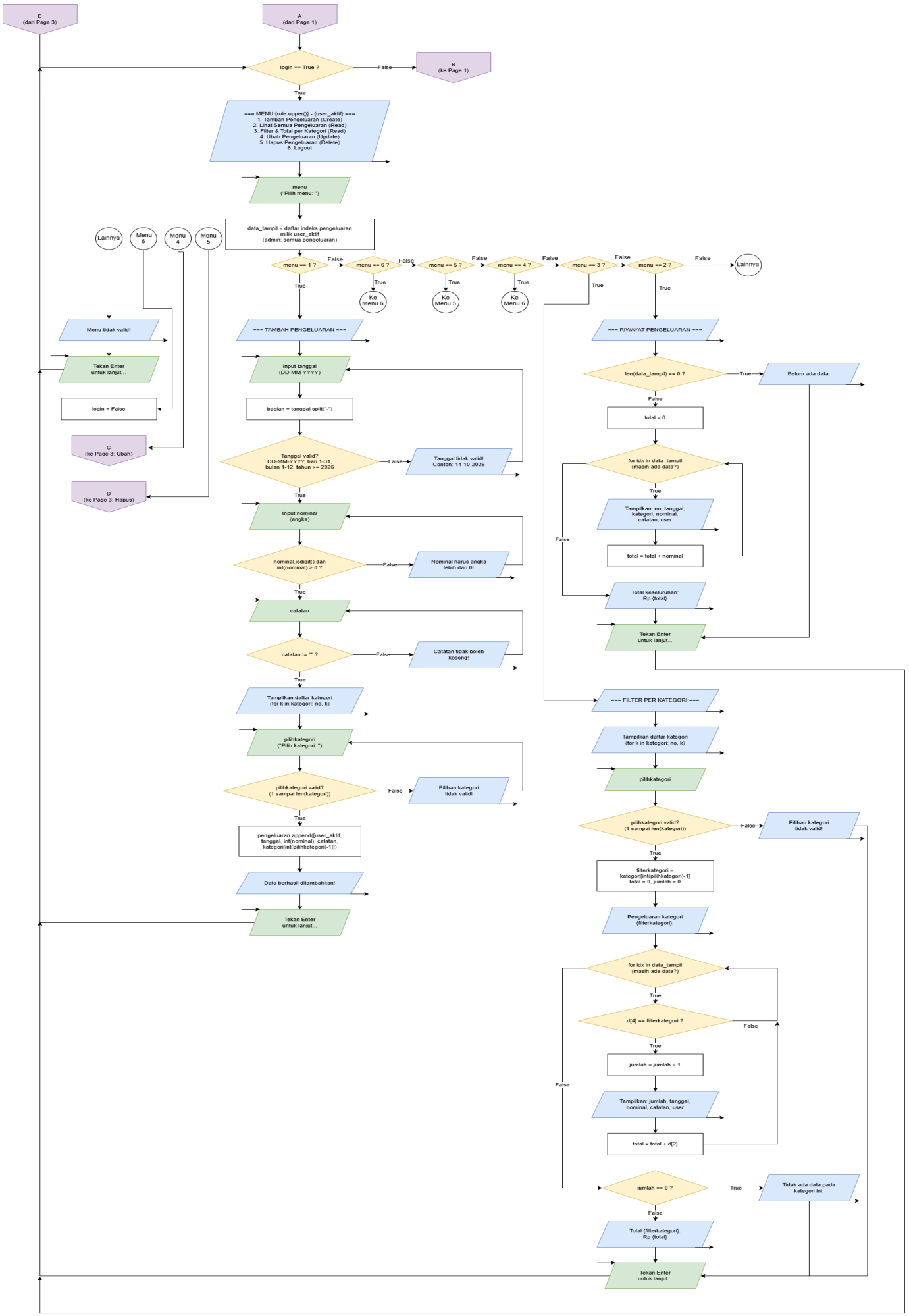
2026

1. Flowchart

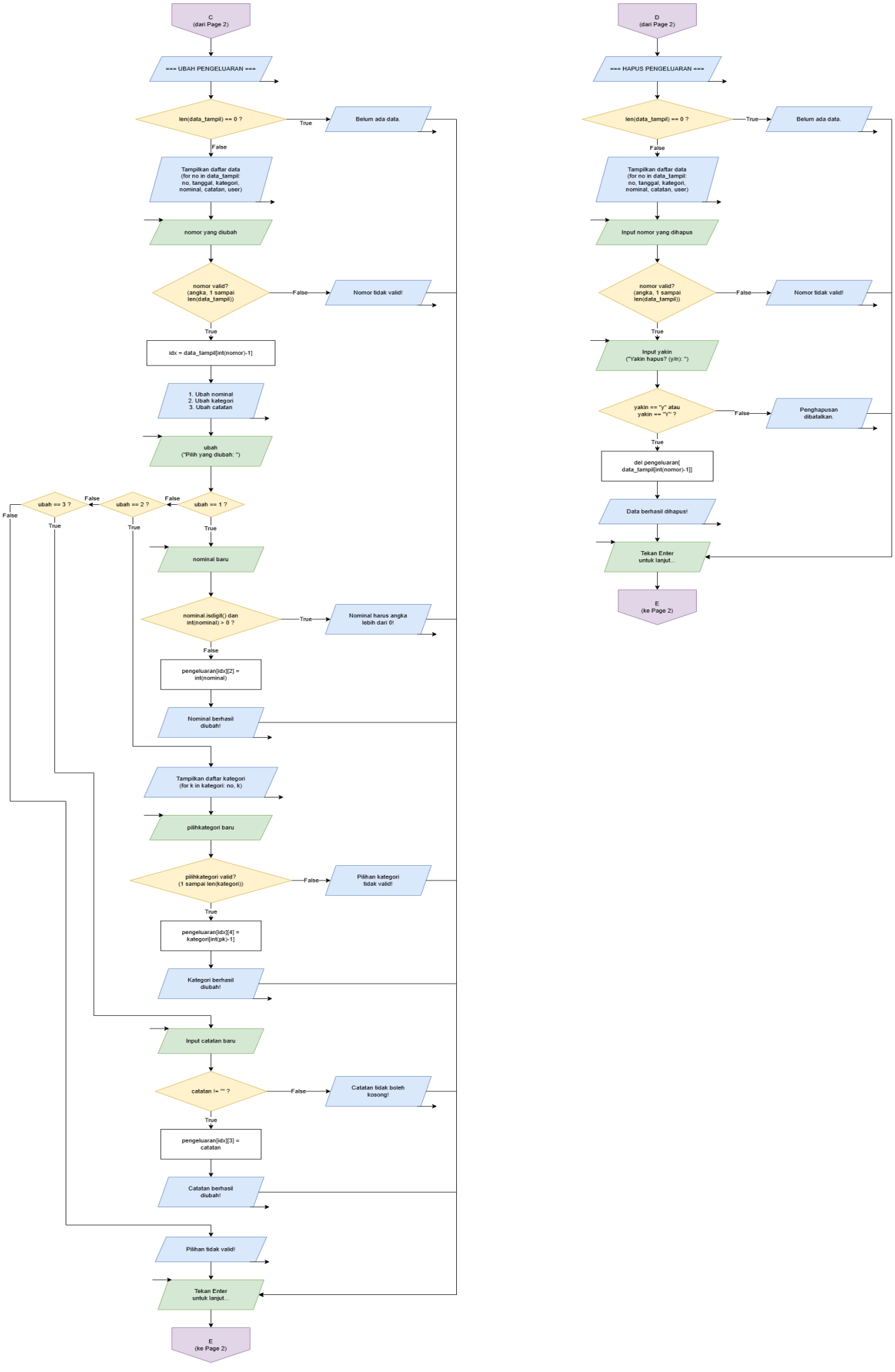

START
user = [["admin","admin123","admin"],
["belva","belva005","user"]]
pengeluaran = []
kategori = ("ATK","Makanan",
"Transportasi","Hiburan","Lainnya")
laman = True
laman == True ?
False
END
True
=== SISTEM PENCATAT KEUANGAN HARIAN ===
1. Login
2. Register
3. Keluar
pilihan
("Pilih Menu: ")
pilihan = 2
True
False
pilihan = 1
False
pilihan = 3
False
True
True
Menu tidak valid!
Terima kasih!
percobaan = 0
berhasil = False
user_aktif = ""
role = ""
Tekan Enter
untuk lanjut...
laman = False
percobaan < 3 and
berhasil == False ?
False
True
=== LOGIN ===
username,
password
Gagal login 3 kali,
kembali ke menu awal
for u in user:
jika u[0]==username dan
u[1]==password:
berhasil = True
user_aktif = u[0]
role = u[2]
Tekan Enter
untuk lanjut....
berhasil == False ?
False
Ya
percobaan = percobaan + 1
Username/password salah!
Sisa percobaan: 3 - percobaan



Gambar 1.1 Flowchart Pages 1



Gambar 1.2 Flowchart Pages 2



Gambar 1.3 Flowchart Pages 3

Penjelasan Singkat Algoritma:

1. Mulai
2. Beri nilai awal: user (dictionary berisi admin dan belva beserta password dan role), pengeluaran = {} (dictionary kosong), id_terakhir = 0, kategori (ATK, Makanan, Transportasi, Hiburan, Lainnya), laman = True
3. Selama laman = True, tampilkan menu awal (1. Login, 2. Register, 3. Keluar), lalu input pilihan
4. Jika pilihan = "2", input username_baru dan password_baru
5. Jika username kosong, tampilkan "Username Tidak Boleh Kosong"
6. Jika username_baru sudah ada sebagai key di dictionary user, tampilkan "Username Sudah Terpakai"
7. Jika password kurang dari 4 karakter, tampilkan "Password Minimal 4 Karakter"
8. Jika semua benar, simpan user baru ke dictionary user dengan username sebagai key dan value berisi password dan role "user", tampilkan "Register berhasil! Silakan login.", lalu kembali ke langkah 3
9. Jika pilihan = "1", isi percobaan = 0 dan berhasil = False
10. Selama percobaan < 3 dan berhasil = False, input username dan password
11. Jika username ada di dictionary user dan password cocok dengan user[u]["password"], berhasil = True, simpan user_aktif dan role dari user[u]["role"], tampilkan "Login Berhasil"
12. Jika tidak cocok, percobaan + 1, tampilkan "Username/password salah! Sisa percobaan: (3 - percobaan)", lalu ulangi langkah 10
13. Jika gagal 3 kali, tampilkan "Gagal login 3 kali, kembali ke menu awal." lalu kembali ke langkah 3
14. Jika berhasil, login = True
15. Selama login = True, tampilkan menu (1. Tambah, 2. Lihat, 3. Filter & Total per Kategori, 4. Ubah, 5. Hapus, 6. Logout), lalu input menu
16. Buat dictionary data_tampil (nomor urut sebagai key dan id pengeluaran sebagai value) yang berisi pengeluaran milik user_aktif (admin melihat semua)
17. Jika menu = "1", input tanggal (DD-MM-YYYY). Selama tidak valid, tampilkan "Tanggal tidak valid!" lalu input ulang
18. Input nominal. Selama bukan angka atau tidak lebih dari 0, tampilkan "Nominal harus angka lebih dari 0!" lalu input ulang
19. Input catatan. Selama kosong, tampilkan "Catatan tidak boleh kosong!" lalu input ulang
20. Tampilkan kategori, input pilihkategori. Selama tidak valid, tampilkan "Pilihan kategori tidak valid!" lalu input ulang
21. id_terakhir bertambah 1, simpan data ke dictionary pengeluaran dengan id_terakhir sebagai key dan value berupa dictionary (user, tanggal, nominal, catatan, kategori), tampilkan "Data berhasil ditambahkan!", lalu kembali ke langkah 15
22. Jika menu = "2", jika data_tampil kosong, tampilkan "Belum ada data."
23. Jika ada data, tampilkan semua data sambil menjumlahkan total, lalu tampilkan "Total keseluruhan: Rp (total)". Kembali ke langkah 15

24. Jika menu = "3", tampilkan kategori, input pilihkategori. Jika tidak valid, tampilkan "Pilihan kategori tidak valid!"
25. Jika valid, tampilkan data dengan kategori yang dipilih sambil menjumlahkan total
26. Jika tidak ada data, tampilkan "Tidak ada data pada kategori ini." Jika ada, tampilkan total kategori. Kembali ke langkah 15
27. Jika menu = "4", jika data_tampil kosong, tampilkan "Belum ada data."
28. Jika ada data, tampilkan daftar data, input nomor yang diubah. Jika nomor bukan angka atau tidak ada di key data_tampil, tampilkan "Nomor tidak valid!"
29. Jika valid, tampilkan pilihan (1. Ubah nominal, 2. Ubah kategori, 3. Ubah catatan), lalu input ubah
30. Jika ubah = "1", input nominal baru. Jika valid, nilai key "nominal" diubah. Jika tidak, tampilkan "Nominal harus angka lebih dari 0!"
31. Jika ubah = "2", tampilkan kategori, input kategori baru. Jika valid, nilai key "kategori" diubah. Jika tidak, tampilkan "Pilihan kategori tidak valid!"
32. Jika ubah = "3", input catatan baru. Jika tidak kosong, nilai key "catatan" diubah. Jika kosong, tampilkan "Catatan tidak boleh kosong!"
33. Jika ubah selain 1, 2, 3, tampilkan "Pilihan tidak valid!". Lalu kembali ke langkah 15
34. Jika menu = "5", jika data_tampil kosong, tampilkan "Belum ada data."
35. Jika ada data, tampilkan daftar data, input nomor yang dihapus. Jika nomor bukan angka atau tidak ada di key data_tampil, tampilkan "Nomor tidak valid!"
36. Jika valid, input yakin (y/n). Jika "y" atau "Y", hapus data dari dictionary pengeluaran dan tampilkan "Data berhasil dihapus!". Jika tidak, tampilkan "Penghapusan dibatalkan."
37. Setelah itu kembali ke langkah 15
38. Jika menu = "6", login = False, kembali ke langkah 3
39. Jika menu selain 1 sampai 6, tampilkan "Menu tidak valid!" lalu kembali ke langkah 15
40. Jika pilihan = "3" (Keluar), tampilkan "Terima kasih!" dan laman = False
41. Jika pilihan selain 1, 2, 3, tampilkan "Menu tidak valid!" lalu kembali ke langkah 3
42. Selesai

`2. Deskripsi Singkat Program

Program ini adalah sebuah program dimana digunakan sebagai Sistem Pencatat Keuangan Harian yang dibuat untuk membantu pengguna mencatat dan mengelola pengeluaran sehari hari. Program menyediakan fitur register dan login supaya setiap pengguna bisa memiliki akun sendiri. Terdapat dua jenis pengguna, yaitu admin dan pengguna biasa. Admin dapat melihat serta mengelola seluruh data pengeluaran, sedangkan pengguna biasa hanya dapat melihat dan mengelola data pengeluaran miliknya sendiri. Program juga membatasi percobaan login maksimal tiga kali untuk mengurangi kesalahan saat memasukkan username dan password.

Program ini menerapkan konsep CRUD (Create, Read, Update, Delete) dengan menggunakan dictionary dan nested dictionary untuk menyimpan serta mengelola data. Fitur Create digunakan untuk menambahkan transaksi baru dengan memasukkan tanggal, nominal, catatan, dan kategori pengeluaran. Fitur Read digunakan untuk melihat riwayat pengeluaran, menghitung total pengeluaran, serta menyaring transaksi berdasarkan kategori, seperti ATK, Makanan, Transportasi, Hiburan, dan Lainnya. Fitur Update digunakan untuk mengubah nominal, kategori, atau catatan transaksi yang sudah tersimpan. Sementara itu, fitur Delete digunakan untuk menghapus transaksi yang tidak diperlukan atau salah dimasukkan, dengan konfirmasi sebelum penghapusan dilakukan.

3. Source Code

A. Fitur Input Data

Fitur input digunakan untuk menerima data yang dimasukkan pengguna, seperti username, password, tanggal transaksi, nominal pengeluaran, catatan, kategori, dan pilihan menu.

```
username_baru = input("Masukkan Username Baru: ")
password_baru = input("Masukkan Password Baru: ")
tanggal = input("Tanggal (DD-MM-YYYY): ")
nominal = input("Nominal (angka): ")
catatan = input("Catatan: ")
pilihkategori = input("Pilih kategori: ")
```

B. Fitur Percabangan (IF/ELIF/ELSE)

Percabangan digunakan untuk menentukan tindakan program berdasarkan kondisi tertentu. Pada program ini, percabangan digunakan untuk memeriksa pilihan menu, memvalidasi input, menentukan hak akses admin atau pengguna biasa, serta menentukan operasi CRUD yang akan dijalankan.

```
  if pilihan == "2":
        os.system('cls')
        print("=== REGISTER ===")
        username_baru = input("Masukkan Username Baru: ")
        password_baru = input("Masukkan Password Baru: ")
        ada = False
        for u in user :
            if u == username_baru:
                ada = True
        if username_baru == "":
            print("Username Tidak Boleh Kosong")
        elif ada == True:
            print ("Username Sudah Terpakai")
        elif len(password_baru) < 4:
            print("Password Minimal 4 Karakter")
        else :
            user[username_baru] = {"password": password_baru, "role":
"user"}
            print("Register berhasil! Silakan login.")
        input("Tekan Enter Untuk Lanjut...")

    elif pilihan == "1":
        percobaan = 0
        berhasil = False
        user_aktif = ""
        role = ""
```

```
        while percobaan < 3 and berhasil == False:
            os.system('cls')
            print("=== LOGIN ===")
            username = input("Username: ")
            password = input("Password: ")

            for u in user:
                if u == username and user[u]["password"] == password:
                    berhasil = True
                    user_aktif = u
                    role = user[u]["role"]
                    print("Login Berhasil")
                    input("Tekan Enter Untuk Lanjut...")
                    break

            if berhasil == False:
                percobaan += 1
                print(f"Username/password salah! Sisa percobaan: {3 -
percobaan}")
                input("Tekan Enter untuk lanjut...")

        login = berhasil
        if not berhasil:
            print("Gagal login 3 kali, kembali ke menu awal.")
            input("Tekan Enter untuk lanjut...")
    elif pilihan == "3":
        print("Terima kasih!")
        laman = False

    else:
        print("Menu tidak valid!")
        input("Tekan Enter untuk lanjut...")
```

C. Fitur Perulangan (FOR dan WHILE)

Perulangan for digunakan untuk memeriksa daftar akun, menampilkan kategori, dan membaca data pengeluaran. Sementara itu, perulangan while digunakan untuk menampilkan menu secara berulang, mengulang proses login selama kesempatan masih tersedia, serta meminta pengguna memperbaiki input yang tidak valid.`while laman == True:`

```
while laman == True:
    os.system('cls')
    print("=== SISTEM PENCATAT KEUANGAN HARIAN ===")
    print("1. Login")
```

```
    print("2. Register")
    print("3. Keluar")
    pilihan = input("Pilih Menu: ")

while login:
    os.system('cls')
    print(f"=== MENU {role.upper()} - {user_aktif} ===")
    print("1. Tambah Pengeluaran")
    print("2. Lihat Semua Pengeluaran")
    print("3. Filter & Total per Kategori")
    print("4. Ubah Pengeluaran")
    print("5. Hapus Pengeluaran")
    print("6. Logout")
    menu = input("Pilih menu: ")

    data_tampil = {}
    nomor_urut = 1
    for id_data in pengeluaran:
        if role == "admin" or pengeluaran[id_data]["user"] == user_aktif:
            data_tampil[nomor_urut] = id_data
            nomor_urut += 1

while valid == False:
    tanggal = input("Tanggal (DD-MM-YYYY): ")
    bagian = tanggal.split("-")
    if len(bagian) == 3:
        if bagian[0].isdigit() and bagian[1].isdigit() and
bagian[2].isdigit():
            if len(bagian[0]) == 2 and len(bagian[1]) == 2 and
len(bagian[2]) == 4:
                hari = int(bagian[0])
                bulan = int(bagian[1])
                tahun = int(bagian[2])
                if 1 <= hari <= 31 and 1 <= bulan <= 12 and tahun >= 2026:
                    valid = True
    if not valid:
        print("Tanggal tidak valid! Contoh: 14-10-2026")

valid = False
while not valid:
    nominal = input("Nominal (angka): ")
    if nominal.isdigit() and int(nominal) > 0:
        valid = True
    else:
        print("Nominal harus angka lebih dari 0!")

valid = False
while not valid:
```

```
    catatan = input("Catatan: ")
    if catatan != "":
        valid = True
    else:
        print("Catatan tidak boleh kosong!")

print("Kategori:")
for i in range(len(kategori)):
    print(f"{i + 1}. {kategori[i]}")
valid = False
while not valid:
    pilihkategori = input("Pilih kategori: ")
    if pilihkategori.isdigit() and 1 <= int(pilihkategori) <= len(kategori):
        valid = True
    else:
        print("Pilihan kategori tidak valid!")
```

D. Fitur Dictionary, Tuple, dan Nested Dictionary

Dictionary disini digunakan untuk menyimpan data ke dalam bentuk pasangan key dan value. Nested dictionary sendiri merupakan dictionary yang berisi dictionary lain, sehingga satu data dapat menyimpan beberapa informasi sekaligus dan diakses menggunakan nama key, bukan nomor indeks. Pada program ini, data akun menggunakan username sebagai key, sedangkan value-nya berisi password dan role. Data pengeluaran menggunakan id transaksi sebagai key, sedangkan value-nya berisi pemilik transaksi, tanggal, nominal, catatan, dan kategori. Dictionary juga digunakan pada data_tampil untuk memetakan nomor urut tampilan ke id pengeluaran.

Tuple digunakan untuk menyimpan daftar kategori pengeluaran yang tersedia, yaitu ATK, Makanan, Transportasi, Hiburan, dan Lainnya.

```
user = {
    "admin": {"password": "admin123", "role": "admin"},
    "belva": {"password": "belva005", "role": "user"}
}

pengeluaran = {}
id_terakhir = 0

kategori = ("ATK", "Makanan", "Transportasi", "Hiburan", "Lainnya")
```

E. Fitur CRUD

Fitur CRUD merupakan bagian utama program untuk mengelola data pengeluaran. Operasi Create dilakukan dengan menambahkan pasangan key dan value baru ke dalam dictionary pengeluaran. Operasi Read dilakukan dengan menampilkan riwayat pengeluaran dan menghitung total nominal, termasuk berdasarkan kategori tertentu. Operasi Update dilakukan dengan mengubah value pada key tertentu di dalam nested dictionary sesuai data yang dipilih. Operasi Delete dilakukan dengan menghapus pasangan key dan value dari dictionary menggunakan del.

1. Create (Tambah Data)

```
id_terakhir += 1
pengeluaran[id_terakhir] = {
    "user": user_aktif,
    "tanggal": tanggal,
    "nominal": int(nominal),
    "catatan": catatan,
    "kategori": kategori[int(pilihkategori) - 1]
}
```

2. Read (Baca Data)

```
for no in data_tampil:
    d = pengeluaran[data_tampil[no]]
    print(f"{no}. {d['tanggal']} | {d['kategori']} | Rp {d['nominal']} |
{d['catatan']} | oleh {d['user']}")
    total += d["nominal"]
print(f"Total keseluruhan: Rp {total}")
input("Tekan Enter untuk lanjut...")
```

3. Update (Ubah Data)

```
pengeluaran[idx]["nominal"] = int(nominal)
```

Contoh ini digunakan untuk memperbarui nominal pengeluaran. lainnya, seperti kategori dan catatan, juga dapat diubah melalui menu Update.

4. Delete (Hapus Data)

```
del pengeluaran[data_tampil[int(nomor)]]
```

Perintah ini digunakan untuk menghapus transaksi berdasarkan nomor data yang dipilih setelah pengguna memberikan konfirmasi.

F. Fitur Validasi dan Error Handling

Validasi input digunakan untuk mencegah data yang tidak sesuai dimasukkan ke dalam program. Pemeriksaan dilakukan menggunakan percabangan, metode string seperti isdigit(), operator in untuk memeriksa key pada dictionary, serta perulangan while. Dengan cara ini, program dapat menolak nominal yang bukan angka atau kurang dari satu, tanggal dengan format yang tidak sesuai, catatan kosong, nomor data yang tidak tersedia, serta pilihan kategori yang tidak tersedia. Penanganan kesalahan dilakukan tanpa menggunakan try dan except.

```
if nominal.isdigit() and int(nominal) > 0:
    valid = True
else:
    print("Nominal harus angka lebih dari 0!")

if nomor.isdigit() and int(nomor) in data_tampil:
    idx = data_tampil[int(nomor)]
else:
    print("Nomor tidak valid!")
```

G. Fitur Clear Terminal dengan Library OS

Library os digunakan untuk membersihkan terminal agar tampilan program lebih rapi setiap kali berpindah menu. Perintah os.system('cls') digunakan pada terminal Windows.

```
import os

os.system('cls')
```

H. Fitur Output (Print dan F-String)

Fitur output digunakan untuk menampilkan menu, riwayat pengeluaran, total nominal, pesan validasi, dan informasi keberhasilan operasi CRUD. F-string memungkinkan program menampilkan nilai variabel secara langsung di dalam teks.

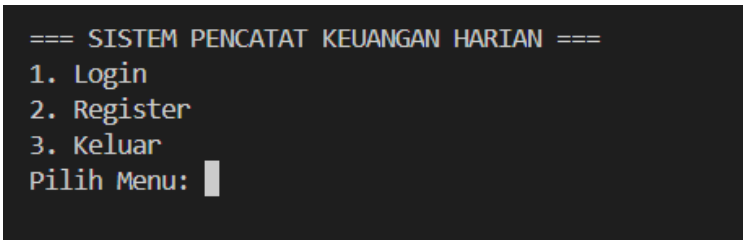
```
print(f"Total keseluruhan: Rp {total}")
print("Data berhasil ditambahkan!")
print("Nominal berhasil diubah!")
print("Data berhasil dihapus!")
```

```
print("=== SISTEM PENCATAT KEUANGAN HARIAN ===")
print("1. Login")
```

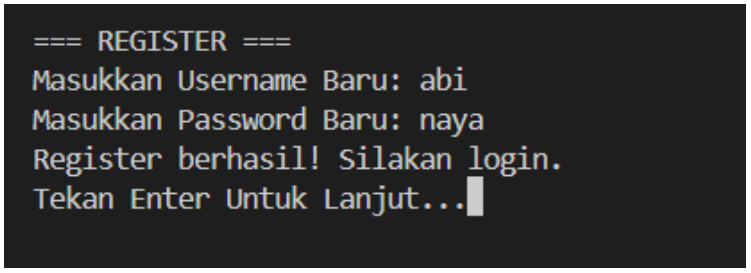
```
print("2. Register")
print("3. Keluar")
```

```
print(f"=== MENU {role.upper()} - {user_aktif} ===")
print("1. Tambah Pengeluaran")
print("2. Lihat Semua Pengeluaran")
print("3. Filter & Total per Kategori")
print("4. Ubah Pengeluaran")
print("5. Hapus Pengeluaran")
print("6. Logout")
```

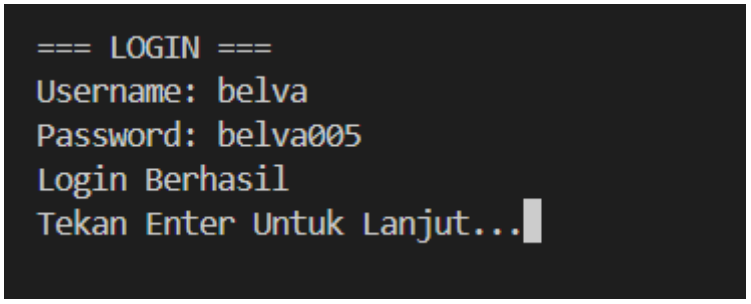
4. Hasil Output



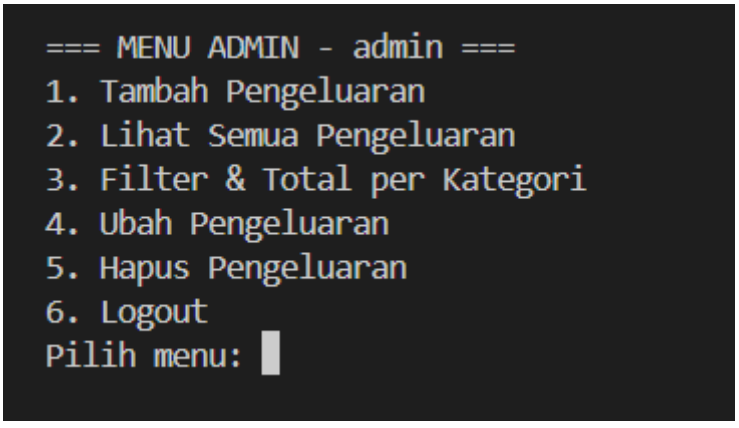
Gambar 4.1 Tampilan Menu Utama



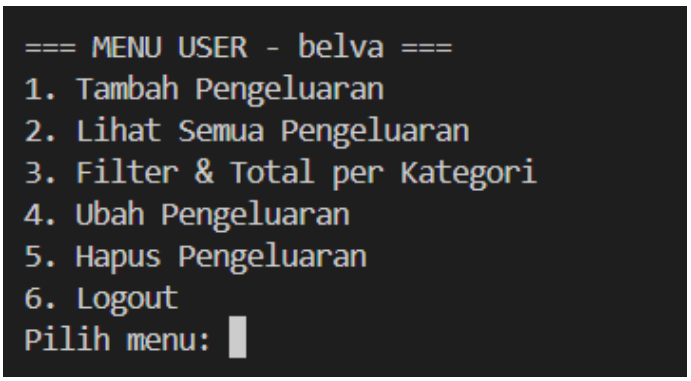
Gambar 4.2 Tampilan Register



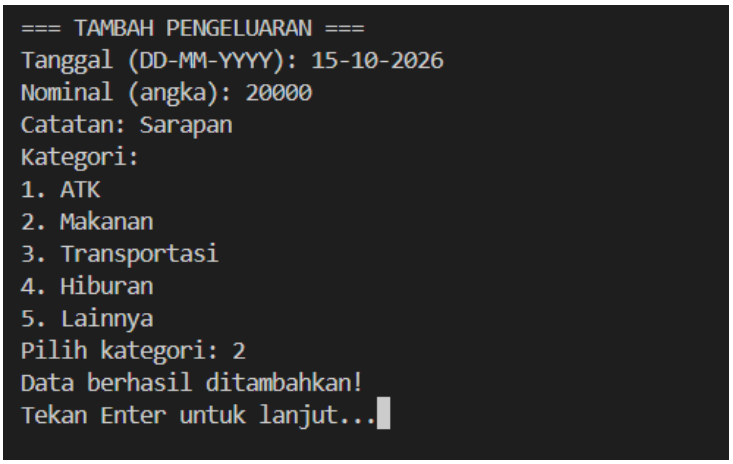
Gambar 4.3 Tampilan Jika Login Berhasil



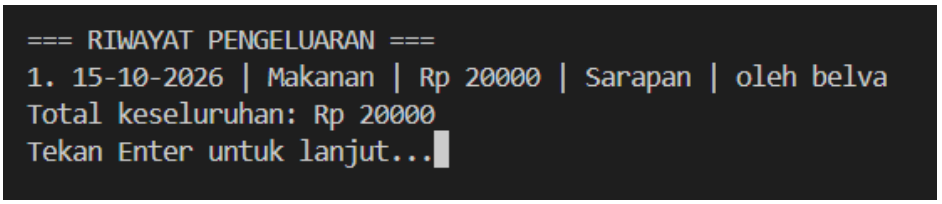
Gambar 4.4 Tampilan Menu Admin



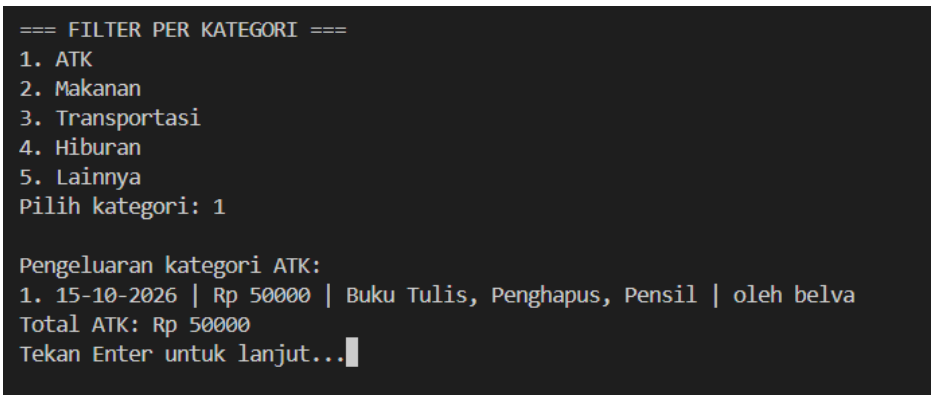
Gambar 4.5 Tampilan Menu Pengguna Biasa



Gambar 4.6 Tampilan Menu Tambah Pengeluaran



Gambar 4.7 Tampilan Menu Riwayat Pengeluaran



Gambar 4.8 Tampilan Menu Filter dan Total per Kategori

```
=== UBAH PENGELUARAN ===
1. 15-10-2026 | Makanan | Rp 20000 | Sarapan | oleh belva
2. 15-10-2026 | ATK | Rp 50000 | Buku Tulis, Penghapus, Pensil | oleh belva
Pilih nomor data yang diubah: 1
1. Ubah nominal
2. Ubah kategori
3. Ubah catatan
Pilih yang diubah: 1
Nominal baru: 15000
Nominal berhasil diubah!
Tekan Enter untuk lanjut...
```

Gambar 4.9 Tampilan Menu Ubah Pengeluaran

```
=== HAPUS PENGELUARAN ===
1. 15-10-2026 | Makanan | Rp 15000 | Sarapan | oleh belva
2. 15-10-2026 | ATK | Rp 50000 | Buku Tulis, Penghapus, Pensil | oleh belva
Pilih nomor data yang dihapus: 2
Yakin hapus? (y/n): y
Data berhasil dihapus!
Tekan Enter untuk lanjut...
```

Gambar 4.10 Tampilan Menu Hapus Pengeluaran

```
=== MENU USER - belva ===
1. Tambah Pengeluaran
2. Lihat Semua Pengeluaran
3. Filter & Total per Kategori
4. Ubah Pengeluaran
5. Hapus Pengeluaran
6. Logout
Pilih menu: 7
Menu tidak valid!
Tekan Enter untuk lanjut...
```

Gambar 4.11 Output Validasi Input Tidak Valid

```
=== SISTEM PENCATAT KEUANGAN HARIAN ===
1. Login
2. Register
3. Keluar
Pilih Menu: 3
Terima kasih!
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026>
```

Gambar 4.12 Output Logout

5.1 Git Add

Perintah git add digunakan untuk menambahkan file baru atau perubahan file ke dalam staging area Git sebelum disimpan melalui commit. Perintah ini dapat digunakan untuk memilih file tertentu atau menambahkan semua perubahan.

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026> git add .
```

Gambar 5.1 Git Add

5.2 Git Commit

Perintah git commit digunakan untuk menyimpan perubahan yang sudah dimasukkan ke staging area sebagai sebuah riwayat perubahan di repositori lokal. Setiap commit dapat diberi pesan untuk menjelaskan perubahan yang dilakukan.

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026\praktikum-apd-2026> git commit -m "POSTTEST 6"
[main 634e550] POSTTEST 6
 2 files changed, 267 insertions(+)
 create mode 100644 A1/Fungsi dan Prosedur.py
 create mode 100644 posttest/posttest-apd-6/2609106005-BelvaAbhinaya-PT-6.py
```

Gambar 5.2 Git Commit

5.3 Git Push

Perintah git push digunakan untuk mengirimkan commit dari repositori lokal ke repositori jarak jauh, misalnya GitHub. Dengan perintah ini, perubahan program dapat diunggah dan disimpan di repositori online.

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026\praktikum-apd-2026> git push
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 2.69 KiB | 919.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/BelvaAbhinaya005/praktikum-apd-2026.git
   35a2739..634e550  main -> main
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026\praktikum-apd-2026>
```

Gambar 5.3 Git Push